

Your network, trained on 1,347 real images

The 2–3–1 network grows to 64–16–1 and meets real data: handwritten 8×8 digits, even versus odd. Loading and plotting are given. The loop — epochs, shuffling, mini-batches, the update — is yours.

Scaffolded lab

22 minutes

Pure NumPy

load_digits

Before you start

Setup · given — the data

Setup · given — the...

1 · predict the starting loss

2 · write the loop

3 · choose the learning...

4 · a number to be proud...

Reflection

If you finish early

Before you start

Open the provided `lab11.ipynb`. The network and Lab 10's backward pass are handed out in the second cell, so the lab does not depend on your Lab 10 having run.



By the end of this lab you can

- Write a complete mini-batch training loop: epochs, shuffling, batches, updates.
- Record and read a training-loss curve and a test-accuracy curve.
- Choose a learning rate from evidence rather than habit.
- Compare the result with a trivial baseline.

LAB 11

Setup · given — the data

Each image is 8×8 grey pixels, flattened to 64 numbers. The label is a yes/no question the network from Lab 10 can already answer: **is this digit even?**

given

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data / 16.0          # 1797 images, 64 pixels each,
                               # scaled to 0-1
y = (digits.target % 2 == 0).astype(float).reshape(-1, 1)  # 1 = even digit, 0 = odd digit

order = np.random.default_rng(0).permutation(len(X))
X_train, y_train = X[order[:1347]], y[order[:1347]]
X_test, y_test = X[order[1347:]], y[order[1347:]]
print("train", X_train.shape, " test", X_test.shape, " share of even digits:",
      round(y_train.mean(), 3))

fig, ax = plt.subplots(1, 8, figsize=(10, 1.6))
for i in range(8):
    ax[i].imshow(X_train[i].reshape(8, 8), cmap="gray_r"); ax[i].axis("off")
    ax[i].set_title("even" if y_train[i, 0] else "odd", fontsize=9)
plt.show()
```

what it should print

OUTPUT

```
train (1347, 64) test (450, 64) share of even digits: 0.483
```

LAB 11

Setup · given — the network

Same functions as Lab 10, with 64 inputs and 16 hidden units. **Read backward once** — it is your Lab 10, with nothing new in it.

given

PYTHON

```

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def init_params(hidden=16, seed=7):
    rng = np.random.default_rng(seed)
    return {"W1": rng.normal(0, 0.1, (64, hidden)), "b1": np.zeros((1, hidden)),
            "W2": rng.normal(0, 0.1, (hidden, 1)), "b2": np.zeros((1, 1))}

def forward(p, X):
    A1 = sigmoid(X @ p["W1"] + p["b1"])
    A2 = sigmoid(A1 @ p["W2"] + p["b2"])
    return A1, A2

def backward(p, X, y):
    # Lab 10's backward pass, unchanged
    A1, A2 = forward(p, X)
    dZ2 = (A2 - y) / len(X)
    dZ1 = (dZ2 @ p["W2"].T) * A1 * (1 - A1)
    return {"W1": X.T @ dZ1, "b1": dZ1.sum(axis=0, keepdims=True),
            "W2": A1.T @ dZ2, "b2": dZ2.sum(axis=0, keepdims=True)}

def bce(p, X, y):
    _, A2 = forward(p, X)
    return float(-np.mean(y * np.log(A2 + 1e-12) + (1 - y) * np.log(1 - A2 + 1e-12)))

def accuracy(p, X, y):
    _, A2 = forward(p, X)
    return float(np.mean((A2 > 0.5) == y))

```

TASK 1 · 2 MINUTES

predict the starting loss

1

Before running: the weights are tiny, so every output is close to 0.5. From Lab 8, what loss does that give? Write your number, then run.

given

PYTHON

```

p0 = init_params()
print("starting loss:", round(bce(p0, X_train, y_train), 4), " starting accuracy:",
      round(accuracy(p0, X_test, y_test), 3))

```

what it should print

OUTPUT

```
starting loss: 0.6945    starting accuracy: 0.409
```

TASK 2 · 10 MINUTES · THE MAIN TASK

write the loop

2

Fill in `train`. For every epoch:

1. shuffle — `rng.permutation(len(X_train))` gives a fresh order of the row numbers;
2. walk through that order in slices of `batch_size`;
3. for each slice, compute the gradients with `backward` and move **every** parameter;
4. after the epoch, append the training loss and the test accuracy.



☛ **Shuffle the row numbers, not the arrays.** Index `X_train` and `y_train` with the same slice of `idx`. Shuffling one without the other is a bug that runs silently — it is one of the four you will hunt in Lab 13.

your turn

PYTHON

```
def train(lr, batch_size, epochs, seed=1):
    p = init_params()
    rng = np.random.default_rng(seed)
    losses, accs = [], []
    for epoch in range(epochs):
        idx = None                                # TODO: a fresh order of row numbers
        for start in range(0, len(X_train), batch_size):
            b = None                              # TODO: this batch's row numbers
            g = None                              # TODO: gradients on this batch
            # TODO: update every parameter in p
            losses.append(bce(p, X_train, y_train))
            accs.append(accuracy(p, X_test, y_test))
        return p, losses, accs

params, losses, accs = train(lr=1.0, batch_size=32, epochs=40)
print("final training loss:", round(losses[-1], 4), " test accuracy:", round(accs[-1], 3))
assert accs[-1] >= 0.95, "check the shuffle, the slice, and that every parameter is updated"
```

one version that works

PYTHON

```
def train(lr, batch_size, epochs, seed=1):
    p = init_params()
    rng = np.random.default_rng(seed)
    losses, accs = [], []
    for epoch in range(epochs):
        idx = rng.permutation(len(X_train))
        for start in range(0, len(X_train), batch_size):
            b = idx[start:start + batch_size]
            g = backward(p, X_train[b], y_train[b])
            for k in p:
                p[k] -= lr * g[k]
            losses.append(bce(p, X_train, y_train))
            accs.append(accuracy(p, X_test, y_test))
    return p, losses, accs

params, losses, accs = train(lr=1.0, batch_size=32, epochs=40)
print("final training loss:", round(losses[-1], 4), " test accuracy:", round(accs[-1],
3))
assert accs[-1] ≥ 0.95
```

what it should print

OUTPUT

```
final training loss: 0.0402 test accuracy: 0.976
```

TASK 3 · 6 MINUTES

choose the learning rate from evidence

3

The plotting is given. Run four learning rates for 40 epochs each and look at **both** panels before you decide.

given

PYTHON

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
for lr in [0.01, 0.1, 1.0, 5.0]:
    _, L, A = train(lr=lr, batch_size=32, epochs=40)
    ax[0].plot(L, label=f"lr = {lr}"); ax[1].plot(A, label=f"lr = {lr}")
    print(f"lr = {lr:<5} final loss {L[-1]:.4f}    test accuracy {A[-1]:.3f}")
ax[0].set_yscale("log"); ax[0].set_xlabel("epoch"); ax[0].set_ylabel("training loss");
ax[0].legend(); ax[0].set_title("Loss")
ax[1].set_xlabel("epoch"); ax[1].set_ylabel("test accuracy"); ax[1].legend();
ax[1].set_title("Accuracy")
plt.show()
```

what it should print

OUTPUT

```
lr = 0.01  final loss 0.6554    test accuracy 0.767
lr = 0.1   final loss 0.2067    test accuracy 0.902
lr = 1.0   final loss 0.0402    test accuracy 0.976
lr = 5.0   final loss 0.1097    test accuracy 0.933
```

Which learning rate would you use, and what in the **curves** — not just the last number — tells you? One or two sentences.

Your answer:

TASK 4 · 2 MINUTES

a number to be proud of, honestly

4

A model is only as impressive as the baseline it beats. Compute the accuracy of always guessing the more common class on the test set.

your turn

PYTHON

```
baseline = None      # TODO: accuracy of predicting the majority training class for every test
                        image
print("baseline:", round(baseline, 3), "    your network:", round(accs[-1], 3))
```

one version that works

PYTHON

```
majority = float(y_train.mean() ≥ 0.5)
baseline = float(np.mean(y_test == majority))
print("baseline:", round(baseline, 3), "    your network:", round(accs[-1], 3))
```

what it should print

OUTPUT

```
baseline: 0.467    your network: 0.976
```

Reflection



Write it in the notebook, in your own words

Your Lab 10 network had 13 parameters and 4 training examples. This one has 1,057 parameters and 1,347 examples. What had to be added to the loop to cope with the larger dataset, and why is one step per **batch** better than one step per **epoch**?

Your answer:

If you finish early

A. Train with `batch_size=1` and `batch_size=1347` (the whole set) for 10 epochs each. Time both with `time.perf_counter()` and compare the curves.

B. Show the test images your network gets wrong, with their true digit (`digits.target[order[1347:]]`). Are the mistakes ones a person would forgive?

your turn

PYTHON

```
# TODO (optional extensions A, B)
```